

GitHub 와 자동차 관리 클래스 프로젝트

2026964038 이지환



<https://github.com/ksuljh/ProgramIntro>

INDEX

01 GitHub – Week 1 ~ 8 

02 자동차 관리 클래스 프로젝트 

03 느낀점 & 나의 예상점수 

Week 1 파이썬 소개

📖 제1장: 파이썬 소개 (Introduction to Python)

📌 1. 컴퓨터와 프로그램의 이해

- **컴퓨터의 유연성:** 컴퓨터는 하드웨어 자체로는 단순한 기계지만, 어떤 ****소프트웨어(프로그램)****를 설치하느냐에 따라 문서 작업, 게임, 인공지능 등 다양한 역할을 수행할 수 있는 유연성을 가집니다.
- **프로그램 (Program):** 컴퓨터가 수행할 작업을 순서대로 적어 놓은 '자세한 명령어(Instruction)들의 리스트'이자 작업 지시서입니다.

📖 2. 프로그래밍 언어와 통역사

- **기계어와 인간의 언어:** 컴퓨터는 오직 0과 1로 이루어진 **기계어**만 이해합니다. 따라서 사람이 컴퓨터에게 명령을 내리기 위해서는 중간 다리 역할을 하는 **프로그래밍 언어**(파이썬, C, 자바 등)가 필요합니다.
- **인터프리터와 컴파일러:** 우리가 작성한 코드를 컴퓨터가 알아들을 수 있게 기계어로 번역해 주는 '통역 소프트웨어'입니다.

💡 3. 파이썬(Python)의 주요 특징

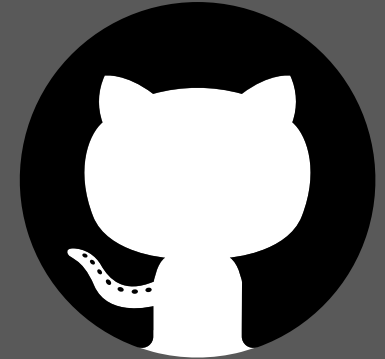
파이썬은 1991년 귀도 반 로섬(Guido van Rossum)이 개발한 언어로, 다음과 같은 강력한 장점을 가집니다.

1. **간결하고 쉬운 문법 (높은 생산성)**
 - C언어나 자바 등 다른 언어에 비해 문법이 직관적입니다. 짧은 코드로도 원하는 기능을 구현할 수 있어 초보자가 배우기 아주 좋습니다.
2. **인터프리터 언어**
 - 코드를 통째로 번역하지 않고 **한 줄씩 읽어서 바로 실행**합니다. 내가 짠 코드의 결과를 즉시 확인할 수 있어 학습과 오류 수정에 유리합니다.
3. **풍부한 라이브러리**
 - 전 세계 개발자들이 만들어 둔 수많은 '확장 기능(라이브러리)'을 무료로 사용할 수 있습니다. 인공지능(Keras, scikit-learn), 이미지 처리(OpenCV), 데이터 분석 등 복잡한 작업도 블록 조립하듯 쉽게 구현 가능합니다.

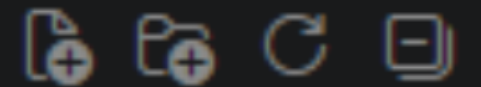
📄 4. 예제 맛보기

화면 출력하기 (print)

```
print("파이썬에 오신 것을 환영합니다!")
print("결과값은", 10, "입니다.")
```



✓ 0306



> 0306

🔗 0306-1.py

🔗 0306-2.py



ksuljh

Edit profile

Popular repositories

Customize your pins

ProgramIntro

Public

Python

programming-study

Forked from daewan1120/programming-study

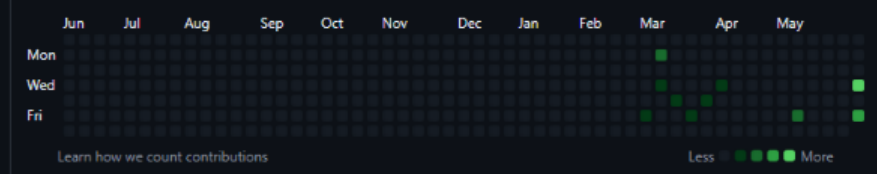
Public

C

33 contributions in the last year

Contribution settings

2026



Week 2 변수와 수식

■ 제2장: 변수와 수식 핵심 요약

- **변수 (Variable):** 값을 저장하는 메모리 공간의 이름입니다. `x = 100` 처럼 값을 대입하는 순간 자동으로 생성되며, 언제든지 새로운 값으로 덮어쓸 수 있습니다.
- **산술 연산자:** 사칙연산(`+`, `-`, `*`, `/`) 외에 거듭제곱(`**`), 몫(`//`), 나머지(`%`)를 계산할 수 있습니다. 연산은 수학처럼 거듭제곱, 곱셈/나눗셈, 덧셈/뺄셈 순서로 진행됩니다.
- **사용자 입력과 형 변환 (`input` , `int` , `float` , `str`):**

 - `input()` : 키보드로 입력을 받는 함수로, 입력된 값은 무조건 ****문자열(str)****로 취급됩니다.
 - `int()` , `float()` : 입력받은 문자열을 수식 계산에 사용하기 위해 각각 **정수**, **실수**로 변환해 줍니다.
 - `str()` : 계산된 숫자를 다시 **문자열**로 변환하여 다른 문장과 자연스럽게 이어 붙일 때 사용합니다.

📄 예제 맛보기

```
# 1. 사용자 입력 및 형 변환 (input, int)
name = input("이름을 입력하세요: ")
age = int(input("나이를 입력하세요: "))

# 2. 산술 연산자를 이용한 수식 계산
next_age = age + 1

# 3. str()을 활용한 문자열 결합 및 출력
print(name + "님의 내년 나이는 " + str(next_age) + "살 입니다.")
```



Week 3 조건문

📖 제3장: 조건문 핵심 요약

- **조건문** (`if`, `elif`, `else`): 주어진 조건이 참(True)인지 거짓(False)인지에 따라 프로그램의 동작을 다르게 제어합니다. `if` 로 시작하며, 추가 조건이 필요할 때는 `elif`, 모든 조건이 거짓일 때 실행할 코드는 `else` 에 작성합니다. 파이썬에서는 들여쓰기로 실행할 코드 구역(블록)을 엄격하게 구분합니다.
- **관계 연산자**: 두 값을 비교하여 참/거짓을 판별합니다. `==` (같다), `!=` (다르다), `>`, `<`, `>=`, `<=` 기호를 사용합니다.
- **논리 연산자**: 여러 개의 조건을 하나로 결합할 때 사용합니다.
 - `and`: 두 조건이 모두 참일 때만 참
 - `or`: 두 조건 중 하나라도 참이면 참
 - `not`: 참과 거짓의 결과를 반대로 뒤집음

📄 예제 맛보기

```
price = 150000
card = "파이썬"

# 관계 연산자, 논리 연산자, 조건문(if-elif-else) 활용
if price >= 100000 and card == "파이썬":
    print("VIP 혜택: 배송비 무료 및 사은품 증정")
elif price > 20000 or card == "파이썬":
    print("일반 혜택: 배송비 무료")
else:
    print("배송비 3000원이 추가됩니다.")
```



Week 4 반복문

🔗 📖 제4장: 반복문 핵심 요약

- **반복문**: 동일하거나 유사한 작업을 자동으로 반복 수행하게 해주는 제어 구조입니다. 파이썬에서는 반복 횟수를 아는 경우 `for` 문을, 특정 조건이 성립하는 동안 반복할 때는 `while` 문을 주로 사용합니다.
- **횟수 반복 (for 문)**: 반복할 횟수가 정해져 있을 때 사용합니다.
 - **리스트와 함께 사용**: `for 변수 in [1, 2, 3]:` 과 같이 리스트의 항목을 하나씩 꺼내며 코드를 반복 실행합니다.
 - **range() 함수**: `range(시작, 종료, 증감)` 을 사용하여 순차적인 정수를 자동으로 생성합니다. (예: `range(5)` 는 0부터 4까지 총 5번 반복).
- **조건 반복 (while 문)**: 지정한 조건식이 참(True)인 동안 계속해서 반복하며, 거짓(False)이 되면 루프를 빠져나옵니다.
 - **무한 루프 주의**: 조건이 영원히 참이 되면 프로그램이 끝나지 않는 무한 루프에 빠집니다. 이를 방지하기 위해 루프 내부에서 조건을 거짓으로 만들 수 있는 코드(예: 변수 증가)를 반드시 포함해야 합니다.
- **중첩 반복문**: 반복문 안에 또 다른 반복문이 포함된 구조입니다. 바깥 반복문이 1번 실행될 때마다 안쪽 반복문은 자신의 반복 횟수를 처음부터 끝까지 모두 실행합니다.

📄 예제 맛보기

```
# 1. for문과 range()를 사용한 횟수 반복 (1부터 5까지의 합 계산)
print("--- for문 ---")
total_sum = 0
for i in range(1, 6): # 1부터 5까지 반복
    total_sum += i
print("1부터 5까지의 합:", total_sum)

# 2. while문을 사용한 조건 반복
print("\n--- while문 ---")
count = 1
while count <= 3:      # count가 3 이하인 동안 반복
    print("현재 카운트:", count)
    count += 1         # 무한 루프를 방지하기 위한 변수 증가

# 3. 중첩 반복문을 활용한 별 찍기 패턴
print("\n--- 중첩 반복문 ---")
for y in range(3):      # 바깥 반복문 (3줄 출력)
    for x in range(5):  # 안쪽 반복문 (한 줄에 별 5개)
        print("*", end=" ")
    print()             # 한 줄 출력이 끝나면 줄바꿈
```


Week 5 함수

■ 제5장: 함수 핵심 요약

- **함수 (Function):** 특정 작업을 수행하는 명령어들의 모음에 이름을 붙인 것입니다. 반복적으로 사용하는 코드를 하나로 묶어두고, 필요할 때마다 호출하여 재사용할 수 있게 해주는 프로그램의 조립 블록 같은 역할을 합니다.
- **함수 정의와 호출:**
 - 파이썬에서는 `def` 함수이름(매개변수): 형식으로 함수를 정의합니다.
 - 함수의 이름과 괄호를 적어주는 것을 **함수 호출**이라고 하며, 호출되면 함수 안의 코드들이 실행된 후 다시 원래 위치로 되돌아옵니다.
- **인수(Argument)와 매개변수(Parameter):** 함수 외부에서 함수로 전달하는 데이터를 '인수'라고 하며, 함수 내부에서 그 값을 받아 사용하는 변수를 '매개변수'라고 합니다.
- **값 반환하기 (`return`):** 함수의 실행이 끝나고 그 결과를 호출한 곳으로 돌려줄 때 `return` 키워드를 사용합니다. 파이썬에서는 `return x, y` 처럼 한 번에 여러 개의 값을 반환하는 것도 가능합니다.
- **지역 변수와 전역 변수:**
 - **지역 변수 (Local Variable):** 함수 안에서 만들어진 변수로, 함수 밖에서는 사용할 수 없고 함수 종료 시 사라집니다.
 - **전역 변수 (Global Variable):** 함수 밖에서 만들어진 변수로, 프로그램 어디서든 값을 읽을 수 있습니다. 단, 함수 안에서 전역 변수의 값을 직접 변경하려면 `global` 변수명 키워드를 먼저 선언해야 합니다.

📖 예제 맛보기

```
# 1. 함수 정의 (매개변수를 받아 면적과 둘레를 계산 후 여러 값 반환)
def calculate_rect(width, height):
    area = width * height
    perimeter = 2 * (width + height)
    return area, perimeter # 두 개의 값을 한 번에 반환

# 2. 전역 변수 선언
w = 10
h = 5

# 3. 함수 호출 및 결과 사용
# 함수에 인수(w, h)를 전달하고, 반환된 결과를 각각의 변수에 저장
rect_area, rect_perimeter = calculate_rect(w, h)

print("사각형의 면적:", rect_area)
print("사각형의 둘레:", rect_perimeter)
```

Week 6 자료구조 I

■ 제6장: 파이썬 자료구조 I (리스트) 핵심 요약

- **리스트 (List):** 여러 개의 항목(데이터)을 순서대로 묶어서 저장하는 컨테이너입니다. 숫자, 문자열 등 다양한 데이터 타입을 함께 저장할 수 있으며, 생성할 때는 대괄호 `[]` 를 사용합니다.
- **인덱스 (Index)와 항목 접근:** 리스트 내의 항목 위치를 나타내는 번호입니다. 항상 **0부터 시작하며**, `리스트명[인덱스]` 형식으로 특정 위치의 값을 읽거나 변경할 수 있습니다. (음수 인덱스 `-1` 을 사용하면 맨 마지막 항목에 접근할 수 있습니다.)
- **리스트 항목 추가 및 삭제:**
 - `append(값)` : 리스트의 맨 끝에 새로운 데이터를 추가합니다.
 - `remove(값)` : 리스트에서 지정한 특정 값을 찾아서 삭제합니다.
- **유용한 내장 함수와 메소드:**
 - `min()` , `max()` : 리스트 내에 있는 값들 중 최소값과 최대값을 쉽게 찾아줍니다.
 - `sort()` : 리스트 안의 항목들을 크기순(오름차순)으로 정렬합니다. 괄호 안에 `reverse=True` 를 넣으면 역순(내림차순)으로 정렬됩니다.

📄 예제 맛보기

```
# 1. 빈 리스트 생성 및 데이터 추가 (append)
scores = []
scores.append(80)
scores.append(70)
scores.append(95)
scores.append(60)

# 2. 인덱스를 활용한 데이터 변경
scores[1] = 75 # 2번째 항목(인덱스 1)의 70을 75로 변경

# 3. 유용한 함수와 메소드 활용
scores.sort() # 오름차순으로 점수 정렬
highest = max(scores) # 최대값 (최고 점수)
lowest = min(scores) # 최소값 (최저 점수)

# 4. 결과 출력
print("정렬된 점수:", scores)
print("최고 점수:", highest)
print("최저 점수:", lowest)
```


Week 7 자료구조 II

🔗 📖 제7장: 파이썬 자료구조 II 핵심 요약

- 튜플 (Tuple): 리스트와 거의 동일하지만 한 번 만들면 값을 변경할 수 없는(Immutable) 자료구조입니다. 생성할 때는 소괄호 `()` 를 사용합니다. 프로그램 실행 중에 실수로라도 값이 바뀌면 안 되는 중요한 데이터를 보호할 때 사용합니다.
- 세트 (Set): 수학의 '집합'과 같은 자료구조로, 데이터의 중복을 허용하지 않고 순서가 없다는 특징이 있습니다. 중복 데이터를 자동으로 제거할 때 유용하며, 기호 `&`, `|`, `-`를 사용하여 교집합, 합집합, 차집합 등의 집합 연산을 쉽게 수행할 수 있습니다. 중괄호 `{}` 나 `set()` 함수를 사용하여 만듭니다.
- 딕셔너리 (Dictionary): **키(Key)와 값(Value)**이 한 쌍으로 묶여 저장되는 자료구조입니다. 사전에서 단어를 찾듯, 인덱스(번호)가 아닌 '키'를 사용하여 연결된 '값'을 매우 빠르게 찾을 수 있습니다. 중괄호 `{}` 안에 `{"키": "값"}` 형태로 작성하며, `[]` 연산자로 값을 추가하거나 `pop()` 으로 삭제할 수 있습니다.
- 문자열 연산 (String): 문자열의 `split()` 함수를 사용해 문장을 특정 기준(예: 공백)으로 쪼개어 리스트로 만들거나, `join()` 을 이용해 흩어진 문자열을 하나로 합칠 수 있습니다.

📄 예제 맛보기

```
# 1. 튜플 (Tuple): 변경 불가능한 고정 데이터
coordinates = (37.5665, 126.9780) # 서울의 위도, 경도
print("위도:", coordinates[0])

# 2. 세트 (Set): 중복 제거 및 집합 연산
words = ["apple", "banana", "apple", "grape"]
unique_words = set(words) # 리스트를 세트로 변환하여 중복(apple) 자동 제거
print("중복이 제거된 단어들:", unique_words)

# 3. 딕셔너리 (Dictionary): 키(Key)와 값(Value) 추가 및 탐색
contacts = {"KIM": "123-4567", "LEE": "987-6543"}
contacts["PARK"] = "555-1111" # 새로운 연락처 추가
contacts.pop("LEE") # 키를 이용해 특정 항목 삭제

# get() 함수를 사용하면 없는 키를 검색해도 예외로 프로그램이 멈추지 않습니다.
phone_number = contacts.get("KIM", "해당 연락처가 없습니다.")
print("KIM의 전화번호:", phone_number)
```



자동차 관리 클래스 코드

```
1 class Car:
2     def __init__(self, model):
3         # 초기화: 이름만 받아서 생성 (기본 연료 60L, 주행거리 0km 시작)
4         self.model = model
5         self.fuel = 60
6         self.mileage = 0
7
8     def drive(self, distance):
9         # 주행: 10km당 1L 소모
10        fuel_needed = distance / 10
11        if self.fuel >= fuel_needed:
12            self.fuel -= fuel_needed
13            self.mileage += distance
14            print(f"🚗 [{self.model}] {distance}km 주행 완료!")
15        else:
16            print(f"⚠️ [{self.model}] 연료가 부족합니다! (현재 연료: {self.fuel:.1f}L)")
17
18    def refuel(self, amount):
19        # 주유: 최대 60L까지만 충전
20        if self.fuel + amount > 60:
21            self.fuel = 60
22            print(f"🛢️ [{self.model}] 연료를 가득 채웠습니다! (60L)")
23        else:
24            self.fuel += amount
25            print(f"🛢️ [{self.model}] {amount}L 주유 완료! (현재: {self.fuel:.1f}L)")
```

```
2 garage = {} # {'자동차이름': 자동차객체} 형태로 저장할 딕셔너리
33
34 while True:
35     print("\n--- [1.등록 | 2.주행 | 3.주유 | 4.차고보기 | 5.종료] ---")
36     menu = input("원하는 작업 번호를 입력하세요: ")
37
38     if menu == "5":
39         print("종료합니다.")
40         break
41
42     # 1. 차량 등록
43     if menu == "1":
44         name = input("새로 등록할 차 이름을 입력하세요: ")
45         garage[name] = Car(name)
46         print(f"🌟 {name}이(가) 등록되었습니다.")
47
48     # 2. 주행하기
49     elif menu == "2":
50         name = input("주행할 차 이름을 입력하세요: ")
51         if name in garage:
52             dist = float(input("주행할 거리(km): "))
53             garage[name].drive(dist)
54         else:
55             print("❌ 등록되지 않은 차량입니다.")
56
57     # 3. 주유하기
58     elif menu == "3":
59         name = input("주유할 차 이름을 입력하세요: ")
60         if name in garage:
61             amt = float(input("주유할 양(L): "))
62             garage[name].refuel(amt)
63         else:
64             print("❌ 등록되지 않은 차량입니다.")
65
66     # 4. 차고 보기 (모든 차량 상태 확인)
67     elif menu == "4":
68         if not garage:
69             print("차고가 비어 있습니다.")
70         else:
71             print("\n📊 --- 현재 차고 상태 ---")
72             for car in garage.values():
73                 print(f"🚗 [{car.model}] 주행: {car.mileage}km | 연료: {car.fuel:.1f}/60L")
```



자동차 관리 클래스 실행

--- [1.등록 | 2.주행 | 3.주유 | 4.차고보기 | 5.종료] ---
원하는 작업 번호를 입력하세요: 1
새로 등록할 차 이름을 입력하세요: 카이엔
🌟 카이엔이(가) 등록되었습니다.

--- [1.등록 | 2.주행 | 3.주유 | 4.차고보기 | 5.종료] ---
원하는 작업 번호를 입력하세요: 2
주행할 차 이름을 입력하세요: 카이엔
주행할 거리(km): 30
🚩 [카이엔] 30.0km 주행 완료!

--- [1.등록 | 2.주행 | 3.주유 | 4.차고보기 | 5.종료] ---
원하는 작업 번호를 입력하세요: 3
주유할 차 이름을 입력하세요: 카이엔
주유할 양(L): 3
🛢️ [카이엔] 3.0L 주유 완료! (현재: 60.0L)

--- [1.등록 | 2.주행 | 3.주유 | 4.차고보기 | 5.종료] ---
원하는 작업 번호를 입력하세요: 4

📊 --- 현재 차고 상태 ---
🚗 [카이엔] 주행: 30.0km | 연료: 60.0/60L

--- [1.등록 | 2.주행 | 3.주유 | 4.차고보기 | 5.종료] ---
원하는 작업 번호를 입력하세요: 5
종료합니다.



느낀점

이번 파이썬 수업에서 변수, 조건문, 반복문, 함수뿐만 아니라 객체와 클래스라는 새로운 개념을 새롭게 알게 되었다. Github 와 VSCode 사용법을 알게되어 프로그래밍을 간편하고 편리하게 할 수 있는 법을 알게 되어 좋았고 변수나 반복문 같은 개념들을 '자동차 관리 클래스'라는 프로젝트로 프로그래밍을 해보면서 파이썬 문법들이 실제 프로그램 속에서 어떻게 돌아가는지 그 흐름을 이해하는데 도움이 되었다. 이런 과정을 통해 내가 정한 프로젝트 프로그램이 의도한 대로 잘 작동하는 모습을 보며 큰 성취감을 느꼈다.

A⁺

나의 예상 점수

GitHub : 14/15

프로젝트 발표 : 15/15

감사합니다